
Minimum Cost Road Network

Connecting All Towns at Minimum Construction Cost
using Prim's and Kruskal's Algorithms

Core Algorithms	Prim, Kruskal, Disjoint Set (Union-Find)
Technologies	Python, Streamlit, Leaflet
Domain	Graph Algorithms / Minimum Spanning Tree

Submitted by	Vallabhaneni Venkata Satya Sai Avula Rithwik Ambati Ashwanth Kunta Harsha Vardhan
Course	CSE Artificial Intelligence and Machine Learning
Institution	Malla Reddy Vishwavidyapeeth

1. Abstract & Introduction

1.1 Abstract

Building roads between towns is expensive, so a planner must connect every town while spending as little as possible. This project models the problem as a **Minimum Spanning Tree (MST)** on a weighted undirected graph in which towns are vertices and candidate roads are edges weighted by construction cost. Two classical greedy algorithms, **Prim's** (priority-queue based) and **Kruskal's** (sort plus Disjoint Set), are implemented from scratch in Python and compared on correctness, number of edges examined, and runtime.

On a sample network of 12 Telangana towns with 66 candidate roads (straight-line distances computed with the Haversine formula), both algorithms select 11 roads totalling **750.43 km**, against 10083.33 km if every candidate road were built. Across 300 randomized correctness tests the two algorithms agreed on the MST cost every time, and benchmarks up to 4,000 towns show Kruskal running slightly faster on the randomly generated sparse and dense graphs tested. The system is designed to display the selected roads on a Leaflet map and to expose the step-by-step decisions of each algorithm through a Streamlit interface.

1.2 Introduction

Infrastructure planning problems such as laying roads, power lines, water pipes, or fibre-optic cables share one structure: a set of sites must be connected, each possible link has a cost, and the cheapest connected layout is wanted. Any cheapest connected layout contains no cycle, because removing one edge of a cycle keeps everything connected and lowers the cost. The optimum is therefore a spanning tree with exactly $n - 1$ edges.

Both Prim's and Kruskal's algorithms solve this optimally using the greedy strategy, yet they behave differently. Prim's grows a single tree outward from a starting town; Kruskal's builds a forest of small trees that merge as cheap edges are accepted. Seeing these two behaviours side by side, rather than only a final cost, is the purpose of this prototype: the user should understand *why* each road was chosen and why others were rejected.

2. Problem Statement, Objectives & Scope

2.1 Problem Statement

Design a road network connecting all towns at minimum construction cost. Compare Prim and Kruskal and visualize the selected roads.

Formally, given a connected, undirected, weighted graph $G = (V, E, w)$, where V is the set of towns and $w(u, v)$ is the cost of building a road between towns u and v , find a subset $T \subseteq E$ that connects all vertices and minimizes the total weight, the sum of $w(e)$ over all e in T .

2.2 Objectives

- Minimize the total construction cost (distance multiplied by a cost-per-kilometre rate) while keeping all towns connected.
- Implement Prim's algorithm (with a binary heap) and Kruskal's algorithm (with a Disjoint Set using path compression and union by rank).
- Compare the two algorithms on the resulting cost, edges examined, and running time for sparse and dense inputs.
- Present the final result clearly: selected roads on a map, the total cost, and an ordered log of accepted and rejected edges that explains each decision.

2.3 Scope

In scope:

- Towns entered manually, loaded from CSV, or taken from a built-in sample dataset, with latitude and longitude.
- Candidate-road costs computed from geographic distance, or supplied directly by the user as an edge list.
- Both algorithms run on the same input, with step logs, MST cost, and performance statistics.
- Map visualization of towns, candidate roads, and the selected roads.

Out of scope (assumptions and limitations):

- Costs are proportional to straight-line (great-circle) distance. Real construction cost also depends on terrain, land acquisition, rivers, and existing roads, which are not modelled.
- The model produces a minimum *total cost* tree. It does not minimize travel time between particular pairs of towns, and it does not add redundancy, so one failed road can disconnect part of the network.
- The network is assumed to be connected. Disconnected input yields a spanning forest (Section 6.3).
- Roads are not allowed to introduce intermediate junction points (Steiner points), which could reduce total length further.

3. Features & Functional Requirements

3.1 Key Features

- **Flexible input:** built-in sample towns, CSV upload (name, latitude, longitude), or a custom edge list with explicit costs.
- **Graph construction:** complete graph from geographic distances, with an optional neighbour threshold to restrict candidate roads.
- **Dual algorithm execution:** Prim and Kruskal run on identical data and results are shown side by side.
- **Explainable steps:** each algorithm records every ADD or SKIP decision with the edge and its cost.
- **Map visualization:** selected roads highlighted over faded candidate roads on an interactive Leaflet map.
- **Comparison dashboard:** total cost, edges examined, runtime, and whether both trees are identical.

3.2 Functional Requirements

ID	Requirement	Priority
FR1	The system shall accept towns with names and coordinates, and edges with costs.	High
FR2	The system shall store the network as an edge list and an adjacency list.	High
FR3	The system shall compute an MST using Prim's algorithm from a user-selected start town.	High
FR4	The system shall compute an MST using Kruskal's algorithm with a Disjoint Set.	High
FR5	The system shall display the total cost and the list of selected roads for each algorithm.	High
FR6	The system shall show an ordered log explaining accepted and rejected edges.	High
FR7	The system shall render the selected roads on a map.	High
FR8	The system shall compare runtime and edges examined for both algorithms.	Medium
FR9	The system shall detect a disconnected network and warn the user.	Medium
FR10	The system shall let the user set a cost-per-km rate to convert distance into a budget.	Low

3.3 Non-Functional Requirements

- **Correctness:** both algorithms must return an MST of identical total cost.
- **Performance:** the sample network and graphs of a few thousand towns should be processed in well under a second.
- **Usability:** results should be understandable without reading code, using plain-language step logs and a visual map.
- **Portability:** runs on any machine with Python 3 and the listed packages.

4. Algorithms & Methodology

4.1 Problem Modelling

Each town becomes a vertex indexed 0 to $n - 1$. Each candidate road becomes an undirected edge (u, v) with weight w . Two representations are kept: an **edge list** of (w, u, v) tuples, convenient for Kruskal's sorting, and an **adjacency list** mapping each town to its (neighbour, weight) pairs, convenient for Prim's expansion. For geographic input the weight is the Haversine great-circle distance:

$$d = 2R * \text{asin}(\sqrt{\sin^2(dlat/2) + \cos(lat1) * \cos(lat2) * \sin^2(dlon/2)}), \quad R = 6371 \text{ km}$$

4.2 Why the Greedy Choice Works

Both algorithms rely on the **cut property**: for any partition of the vertices into two non-empty sets, the cheapest edge crossing the partition belongs to some MST. Prim's applies this to the cut (tree so far, remaining towns). Kruskal's applies it to the cut separating one component from all others. Since every accepted edge is safe by this property, the final tree is optimal.

4.3 Disjoint Set (Union-Find)

Kruskal's needs a fast test for whether adding an edge would create a cycle, which is true exactly when its two endpoints already lie in the same component. The Disjoint Set supports *find*(x), returning the representative of x 's component, and *union*(a, b), merging two components. Two optimizations make operations run in nearly constant amortized time, $O(\alpha(n))$, where α is the inverse Ackermann function: **path compression** (flattening the path during find) and **union by rank** (attaching the shorter tree beneath the taller one).

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n)); self.r = [0]*n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]    # path compression (halving)
            x = self.p[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b: return False              # same component -> cycle
        if self.r[a] < self.r[b]: a, b = b, a    # union by rank
        self.p[b] = a
        if self.r[a] == self.r[b]: self.r[a] += 1
        return True
```

4.4 Kruskal's Algorithm

- Sort all edges by increasing cost.
- Create a Disjoint Set with every town in its own component.
- For each edge (u, v) in sorted order: if $\text{find}(u) \neq \text{find}(v)$, accept the edge and union the two components; otherwise reject it, since it would form a cycle.
- Stop when $n - 1$ edges have been accepted.

```
def kruskal(n, edges):
    # edges = [(w, u, v), ...]
    d = DSU(n); mst = []; cost = 0
    for w, u, v in sorted(edges):
        if d.union(u, v):
            mst.append((u, v, w)); cost += w
```

```

        if len(mst) == n - 1: break
    return mst, cost

```

Complexity: sorting dominates, giving $O(E \log E) = O(E \log V)$, since $\log E \leq 2 \log V$. Union-Find adds only $O(E \alpha(V))$. Space is $O(V + E)$.

4.5 Prim's Algorithm

- Start the tree at a chosen town and push all its incident edges into a min-heap.
- Repeatedly pop the cheapest edge. If its far endpoint is already in the tree, discard it (stale entry).
- Otherwise add the edge and the new town to the tree, and push the new town's edges to unvisited neighbours.
- Stop when $n - 1$ edges have been accepted.

```

def prim(n, adj, start=0):
    # adj[u] = [(v, w), ...]
    seen = [False]*n; seen[start] = True
    h = [(w, start, v) for v, w in adj[start]]; heapq.heapify(h)
    mst = []; cost = 0
    while h and len(mst) < n - 1:
        w, u, v = heapq.heappop(h)
        if seen[v]: continue
        seen[v] = True; mst.append((u, v, w)); cost += w
        for x, wx in adj[v]:
            if not seen[x]: heapq.heappush(h, (wx, v, x))
    return mst, cost

```

Complexity: with a binary heap and adjacency list, $O(E \log V)$. With an adjacency matrix and no heap it is $O(V^2)$, which is better for very dense graphs. Space is $O(V + E)$.

4.6 Comparison of the Approaches

Aspect	Prim's	Kruskal's
Strategy	Grows one tree from a start vertex	Merges a forest of components
Key data structure	Priority queue (min-heap)	Sorted edge list + Disjoint Set
Time (heap / sort)	$O(E \log V)$	$O(E \log V)$
Natural input form	Adjacency list or matrix	Edge list
Typically favoured on	Dense graphs	Sparse graphs
Disconnected graph	Spans only the start component	Produces a spanning forest
Intermediate state	Always a single connected tree	Multiple partial trees

5. Technologies & Implementation

5.1 Technology Stack

Technology	Role in the project
Python 3	Implementation of graph model, Disjoint Set, Prim, Kruskal, and benchmarking. Standard-library <code>heapq</code> provides the priority queue.
Streamlit	Interface layer: input selection, algorithm controls, tables, step logs, and metrics dashboard.
Leaflet (via folium)	Interactive map showing towns, candidate roads, and highlighted selected roads.
Matplotlib	Static figures used in this report (MST maps and performance plots).

5.2 System Architecture

The prototype is organized into three layers so that the algorithms stay independent of the user interface:

- **Data layer:** loads towns and edges, computes Haversine distances, builds the edge list and adjacency list.
- **Algorithm layer:** `DSU`, `kruskal()`, and `prim()`. Each returns the chosen edges, total cost, an ordered step log, and statistics such as edges examined and runtime.
- **Presentation layer:** Streamlit renders the controls and tables, and passes the selected edges to a Leaflet map as coloured polylines.

5.3 Workflow

Step	Action
1	User chooses the sample towns, uploads a CSV, or enters an edge list.
2	System builds the graph, assigns edge costs, and checks connectivity.
3	User selects the algorithm(s) and, for Prim, the start town.
4	Algorithms run; step logs, MST edges, and cost are recorded.
5	Results are displayed as a table, a map, and a comparison panel.

5.4 Implementation Notes

- Both algorithms return a log of (ADD | SKIP, u , v , w) events. This log is what makes the result explainable in the interface.
- Prim's uses lazy deletion: stale heap entries are skipped on pop instead of being decreased in place, which keeps the code simple while preserving $O(E \log V)$.
- Kruskal's stops as soon as $n - 1$ edges are accepted, so it often examines only a fraction of the sorted list (Section 6.1).
- Ties in edge cost can produce different but equally optimal trees. Costs are compared exactly, and the tests compare total cost rather than assuming identical edge sets.
- The algorithm layer and the benchmark and test code in this report were run directly in Python. The Streamlit and Leaflet layer is specified here as the presentation design for the prototype.

6. Results, Testing & Performance

6.1 Sample Network Result

The sample uses 12 towns of Telangana, giving 66 candidate roads (every pair). Distances are great-circle estimates from approximate town-centre coordinates, so they are shorter than real road distances. Both algorithms produced a tree of 11 roads.

Metric	Prim (start: Hyderabad)	Kruskal
Roads selected	11	11
Total length (km)	750.43	750.43
Edges examined / popped	24	24 of 66
Same set of roads?	Yes	Yes
Cost if all candidate roads built (km)	10083.33	10083.33
Illustrative budget at 2.5 crore INR per km (assumed rate)	1876.07 crore	1876.07 crore

The MST uses about 7.4% of the length of the full candidate network. The rate of 2.5 crore INR per km is an assumed placeholder to show the budget feature, not a sourced construction cost.

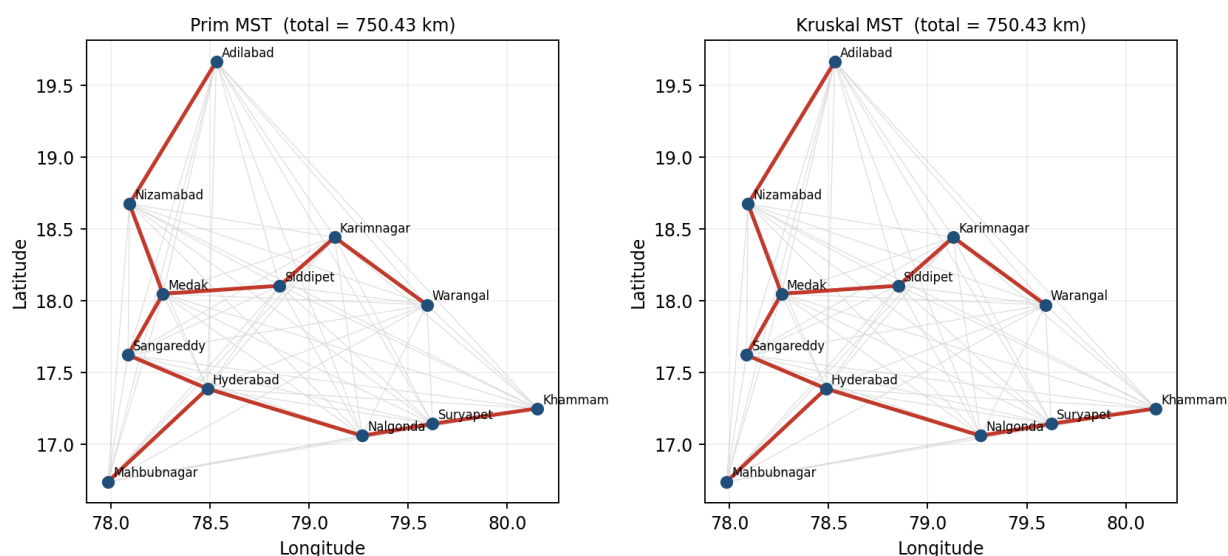


Figure 1. Roads selected by Prim (left) and Kruskal (right); red lines are chosen roads, grey lines are rejected candidates.

6.2 Step-by-Step Explanation (Kruskal)

The table lists the first 14 edges in sorted order, showing how the Disjoint Set decides each one. A SKIP means both towns were already connected through earlier roads, so building the edge would only add cost.

#	Edge	Cost (km)	Decision	Reason
1	Nalgonda – Suryapet	38.95	ADD	Joins two separate components
2	Karimnagar – Siddipet	47.54	ADD	Joins two separate components
3	Hyderabad – Sangareddy	49.76	ADD	Joins two separate components

#	Edge	Cost (km)	Decision	Reason
4	Medak – Sangareddy	51.00	ADD	Joins two separate components
5	Khammam – Suryapet	57.34	ADD	Joins two separate components
6	Medak – Siddipet	62.57	ADD	Joins two separate components
7	Warangal – Karimnagar	71.72	ADD	Joins two separate components
8	Nizamabad – Medak	71.86	ADD	Joins two separate components
9	Hyderabad – Medak	77.23	SKIP	Both towns already connected (cycle)
10	Warangal – Siddipet	79.83	SKIP	Both towns already connected (cycle)
11	Hyderabad – Siddipet	88.60	SKIP	Both towns already connected (cycle)
12	Hyderabad – Mahbubnagar	89.66	ADD	Joins two separate components
13	Hyderabad – Nalgonda	90.52	ADD	Joins two separate components
14	Warangal – Suryapet	92.23	SKIP	Both towns already connected (cycle)

Prim's, started at Hyderabad, selects the same 11 roads but in a different order, because it always extends the one growing tree outward from Hyderabad rather than merging scattered small components.

6.3 Testing

Test	Description	Result
Cross-validation	300 random connected graphs (2–40 towns, varying density, random start vertex for Prim). Compared Prim and Kruskal total cost.	300 / 300 identical
Edge count	Every MST should contain exactly $n - 1$ edges for connected input.	Pass (all cases)
Sample dataset	Both algorithms on the 12-town network.	Both 750.43 km
Disconnected input	6 towns, only 3 edges forming two components plus 1 isolated town ($n - 1 = 5$ would be needed for a tree).	Kruskal: 3 edges (spanning forest). Prim: 2 edges (start component only).

The disconnected case confirms the behavioural difference noted in Section 4.6. The prototype therefore checks connectivity before running and warns the user, as required by FR9.

6.4 Performance Comparison

Random connected graphs were generated with integer weights up to 10^6 . Each point is the mean of 3 runs on different random graphs, measured with Python's `perf_counter`. Kruskal's time includes sorting. Prim's time excludes building the adjacency list. Total MST cost was identical in every run. Absolute times depend on the machine and Python implementation, so only the relative trend should be read from them.

Sparse graphs ($m = 4n$)

Towns (n)	Edges (m)	Kruskal (ms)	Prim (ms)	Prim / Kruskal
200	800	0.34	0.37	1.08x
500	2000	1.16	1.34	1.15x
1000	4000	3.16	3.14	0.99x
2000	8000	6.57	7.15	1.09x
4000	16000	14.19	17.59	1.24x

Denser graphs ($m = 40n$)

Towns (n)	Edges (m)	Kruskal (ms)	Prim (ms)	Prim / Kruskal
200	8000	2.27	2.41	1.06x
500	20000	8.00	9.97	1.25x
1000	40000	22.44	28.79	1.28x
2000	80000	55.61	70.75	1.27x
4000	160000	134.73	204.06	1.51x

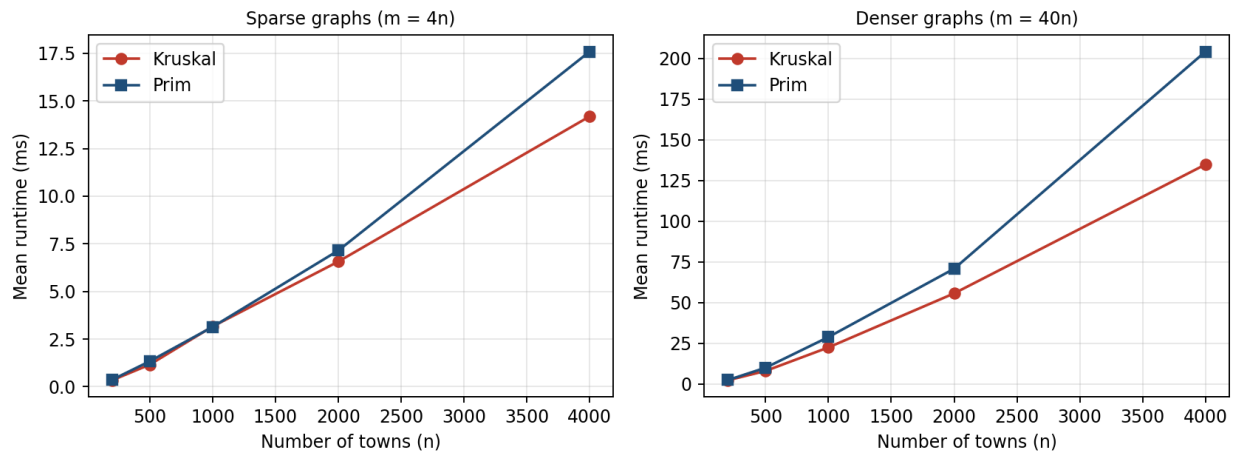


Figure 2. Mean runtime of Prim and Kruskal as the number of towns grows.

6.5 Discussion

- **Both algorithms are correct.** They gave the same total cost in all 300 randomized tests and on the sample network.
- **Kruskal's examined fewer edges than it could have.** On the sample network it stopped after 24 of 66 sorted edges, because the tree was complete before the longest edges were reached.
- **Kruskal's was slightly faster in this experiment.** Its advantage grew on the denser graphs (about 1.5x at $n = 4000$, $m = 160,000$). This is the opposite of the common rule of thumb that Prim's suits dense graphs. That rule applies mainly to the $O(V^2)$ array version of Prim's, whereas the heap version tested here pays for pushing many edges. Python's built-in sort is also highly optimized, which favours Kruskal's.
- **These timings should not be over-generalized.** Results come from one language, one machine, random weights, and graphs of up to 4,000 towns. Real road networks are sparse and geometric, and a Fibonacci-heap or matrix-based Prim could behave differently.
- **Both algorithms explain themselves differently.** Kruskal's log makes cycle rejection easy to see; Prim's log makes the growth of a single connected network easy to follow.

7. Conclusion, Future Scope & References

7.1 Conclusion

This project turned the road-planning requirement into a working algorithmic prototype. The towns and candidate roads are represented as a weighted graph, and the minimum cost network is found as a Minimum Spanning Tree using both Prim's algorithm and Kruskal's algorithm with a Disjoint Set. On the 12-town Telangana sample, the optimal network needs 11 roads totalling 750.43 km, roughly 7.4% of the full candidate network. Both algorithms agree on cost in every test, differ in order of decisions and behaviour on disconnected input, and have the same $O(E \log V)$ bound. In our benchmarks Kruskal's was modestly faster than the heap-based Prim's. Step logs and map visualization make the choice of every road explainable, not just its final cost.

7.2 Limitations

- Straight-line distance is only a proxy for construction cost.
- A minimum-cost tree has no redundancy; failure of one road can isolate towns.
- Benchmarks used synthetic random graphs rather than real road data.

7.3 Future Scope

- Use real cost models: terrain, land cost, river or railway crossings, and existing roads that can be reused at zero cost.
- Import real road-network data (for example OpenStreetMap) and compute road-distance edges instead of straight-line distances.
- Add **Steiner tree** heuristics that allow new junction points to shorten the total network.
- Add redundancy through a second-best MST, or a 2-edge-connected network, for fault tolerance.
- Constrained variants: degree limits at towns, mandatory or forbidden roads, and budget caps.
- Implement array-based and Fibonacci-heap Prim, and Boruvka's algorithm, for a broader comparison.
- Animate the algorithms step by step in the Leaflet map and support very large graphs.

7.4 References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. (Chapter 21: Data Structures for Disjoint Sets; Chapter 21/23: Minimum Spanning Trees.)
- [2] R. C. Prim, "Shortest Connection Networks and Some Generalizations," *Bell System Technical Journal*, 36(6), 1957, pp. 1389–1401.
- [3] J. B. Kruskal, "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem," *Proceedings of the American Mathematical Society*, 7(1), 1956, pp. 48–50.
- [4] R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," *Journal of the ACM*, 22(2), 1975, pp. 215–225.
- [5] S. Dasgupta, C. Papadimitriou, U. Vazirani, *Algorithms*, McGraw-Hill, 2008.
- [6] Python Software Foundation, *heapq – Heap queue algorithm*, Python 3 documentation, docs.python.org.
- [7] Streamlit documentation, docs.streamlit.io; Leaflet documentation, leafletjs.com; folium documentation, python-visualization.github.io/folium.